

Section 2.4: Table Detection & Extraction

Tables present one of the most complex challenges in PDF text extraction. Much like paragraphs and headings, PDF files do not contain explicit table objects (there are no HTML-like <table>, <tr>, or <td> tags in native PDF code).

Instead, a table in a PDF is merely a visual arrangement of isolated text characters positioned near graphical vector lines or aligned across wide whitespace channels.

1. The Table Extraction Problem

When a native PDF parser extracts text from a table without spatial understanding, it typically reads left-to-right across the page width, reading straight across cells on the same vertical baseline:

Visual Table Layout:

Company	Role	Duration
Tech Corp Inc.	Lead Architect	2020 - Present
Cloud Solutions LLC	Systems Engineer	2017 - 2020

Naïve Stream Extraction Output:

"Company Role Duration Tech Corp Inc. Lead Architect 2020 - Present Cloud Solutions LLC Systems Engineer 2017 - 2020"

The tabular structure is erased, merging column values into a single unstructured string.

2. Taxonomy of PDF Tables

To extract tabular data reliably, extraction engines must classify tables into three structural types:

1. Bordered Tables (Lattice/Grid Tables)

Surrounded by explicit horizontal and vertical vector graphics lines (re, m, l PDF path operators).

Header 1	Header 2
Data A	Data B

2. Borderless Tables (Stream Tables)

Contain no vertical or horizontal dividing lines. Cells are defined purely by visual alignment and whitespace gutters.

Company	Role	Year
Tech Corp	Lead	2020
Cloud Systems	Tech	2017
Data Corp	Dev	2015

3. Hybrid / Partial Tables

Use only horizontal divider lines (e.g., under headers or section footers) without vertical column dividers (extremely common in academic papers and modern resumes).

Company	Role	Year
Tech Corp	Lead	2020

3. Algorithmic Extraction Methodologies

Engineers employ two primary deterministic algorithms alongside modern computer vision models

Layout Parsing Processing Flow

The **Raw PDF Page Stream** is evaluated and routed to one of two primary parsing algorithms depending on the presence of structural metadata:

- **Pathway A: Lattice Algorithm (Vector Grid Processing)** *Designed for documents with explicit table borders.*
 - Extract vector line paths (l, m, re).
 - Calculate line intersections (X, Y nodes).
 - Reconstruct grid cells (Bounding boxes).
 - Map text within cell bounding boxes.
- **Pathway B: Stream Algorithm (Whitespace Projection)** *Designed for documents relying on spacing for alignment.*
 - Cluster words by vertical baseline Y-coord.
 - Identify vertical whitespace channels (Gutters).
 - Assign text blocks into column grids.
 - Merge multi-line wrapped text.

A. The Lattice Algorithm (Vector Intersection Parsing)

- **Used by:** Camelot (Lattice mode), pdfplumber, Tabula.
- **Mechanism:**
 - Scans the PDF drawing operators for explicit lines and path primitives.
 - Converts vector paths into image mask arrays or bounding rectangles.
 - Detects intersections where horizontal lines meet vertical lines to form vertex coordinates (x, y).
 - Builds a network of rectangular bounding boxes (cells).
 - Queries text objects that fall spatially inside each cell coordinate block.
- **Strength:** Near 100% accuracy on explicit grid tables, merged cells, and complex multi-line tabular data.
- **Weakness:** Fails completely on borderless or stream tables where vector lines are missing.

B. The Stream Algorithm (Whitespace Projection & Alignment)

- **Used by:** Camelot (Stream mode), pdfplumber, Tabula.
- **Mechanism:**
 - Detects text bounding boxes and groups words sharing similar vertical baselines (rows).
 - Projects text densities along the horizontal axis to locate empty vertical gaps (gutters).
 - Draws implicit column boundary lines down these gutters.
 - Assigns text blocks into the matrix intersection of Row_i and Column_j.
- **Strength:** Works on borderless tables and whitespace-delimited columns.
- **Weakness:** Highly sensitive to column alignment, wide cell spacing, or text wrapping across multiple lines.

C. Deep Learning & Vision Table Transformers

- **Used by:** Adobe PDF Extract, AWS Textract (TABLES feature), Azure Document Intelligence, Table Transformer (TATR), LlamaParse.
- **Mechanism:** Models pass page image embeddings through Object Detection and Bounding Box Prediction heads, generating HTML <table> structures complete with colspan and rowspan tags.

4. Edge Cases & Complex Structural Handling

A. Merged Cells (colspan and rowspan)

- **Problem:** A single header cell spans across three sub-columns, or a category label spans three vertical rows.
- **Solution:** In Lattice mode, cell bounding boxes are evaluated for dimensions exceeding standard unit grid cells. Merged cells inherit parent coordinates and populate empty child slots in the serialized grid matrix.

B. Multi-Line Cell Wrapping vs. New Rows

- **Problem:** A single cell contains wrapped multi-line text (e.g., job descriptions), which can be misidentified as multiple distinct rows.

Row 1 | Tech Corp | Developed microservice architecture for cloud <-- Line 1 of description

		deployment handling 10k requests/sec.	<-- Line 2 (Same Cell!)

Row 2		Data Corp Built ETL data pipelines in Python.	<-- New Row

- Solution:** Inspect vertical line spacing (leading) within the column. If the vertical gap between lines inside a column is smaller than the row margin gap, the lines are concatenated with a space rather than splitting into a new table row.

5. Tool Capabilities Matrix for Tables

Tool / Engine	Primary Algorithm	Bordered Tables	Borderless Tables	Multi-Line Cell Support
Camelot-py	Lattice & Stream	Excellent	Good (Requires tuning)	High
pdfplumber	Custom Line/Word Grids	High	Moderate	Requires custom code
Tabula-py	Java-based Extraction	High	Moderate	Moderate
AWS Textract	Deep Learning Vision	High	High	High (Returns HTML/JSON)
Adobe PDF Extract	Deep Learning / Structural	High	High	High